Spark-MongoDB Project

Technical Documentation

Data Processing with Apache Spark & MongoDB Atlas

Team Members	
Salma Ahmed Hassan	Menna AL-Zahaby
Samira Gamal	Mohab Badawy
Mohamed Tarek	Youssef ELshazly

Repository: github.com/mohamedtarek750/Spark-MongoDB-Project

Date: May 2026

1. Project Overview

This project demonstrates an end-to-end data engineering pipeline that leverages Apache Spark for large-scale distributed data processing and MongoDB Atlas as the cloud-hosted NoSQL storage back-end. The pipeline ingests three structured CSV datasets totalling 30,000 records, performs data cleaning and transformation using PySpark, persists the results into MongoDB collections, and finally runs analytical aggregation queries whose outputs are visualised as statistical charts.

The project was developed as part of a Big Data course assignment and covers the full lifecycle from raw data ingestion through to insight generation, providing practical exposure to industry-standard tools used in modern data engineering.

2. Objectives

- Load and process large CSV datasets using Apache Spark (PySpark).
- Apply data cleaning operations including deduplication and type casting.
- Establish a connection to MongoDB Atlas and persist processed data into separate collections.
- Execute MongoDB aggregation pipeline queries to derive business insights.
- Produce visualisations (bar charts, pie charts, line charts, histograms) from query results.

3. Technology Stack

Technology	Version / Notes	Role in Project
Python	3.x	Primary programming language
Apache Spark	PySpark	Distributed data processing engine
PySpark	pyspark (pip)	Python API for Apache Spark
MongoDB Atlas	Cloud-hosted cluster	NoSQL document database storage
PyMongo	pymongo (pip)	Python driver for MongoDB
Jupyter Notebook	connect.ipynb	Interactive development environment
Matplotlib / Seaborn	via notebook	Data visualisation libraries

4. Repository Structure

The repository is hosted publicly on GitHub and contains the following files:

File / Asset	Description
spark-py.py	Main Python script: Spark session, data loading, cleaning, MongoDB ingestion, and queries

connect.ipynb	Jupyter Notebook: interactive version of the pipeline with visualisation cells (2,106 lines, 375 KB)
requirements.txt	Python dependency list: pymongo, pyspark
customers-10000.csv	Customer dataset — 10,000 records with fields: Index, Customer Id, First Name, Last Name, Co
people-10000.csv	People dataset — 10,000 records with demographic information
organizations-10000.csv	Organisations dataset — 10,000 records with company-level data
Bar Chart.jpeg	Bar chart visualisation — customers per country
Bar Chart (2).jpeg	Bar chart visualisation — customers per company
Pie chart.jpeg	Pie chart visualisation — gender distribution
Line Chart.jpeg	Line chart visualisation — subscription trends over time
Predictive_Churn_Analytics.pdf	Supplementary report on predictive churn analytics
Week2.pdf	Week 2 progress report / assignment deliverable

5. Dataset Description

The project uses three CSV datasets, each containing 10,000 records, sourced from a mock data generator. All three files share a consistent structure and are loaded directly by PySpark.

5.1 Customers Dataset (customers-10000.csv)

Contains transactional and contact information for individual customers.

Field	Data Type	Description
Index	Integer	Sequential record identifier
Customer Id	String	Unique alphanumeric customer identifier
First Name	String	Customer first name
Last Name	String	Customer last name
Company	String	Associated company or employer
City	String	City of residence
Country	String	Country of residence
Phone 1	String	Primary phone number
Phone 2	String	Secondary phone number
Email	String	Customer email address
Subscription Date	Date	Date the customer subscribed (yyyy-MM-dd)
Website	String	Customer or company website URL

5.2 People Dataset (people-10000.csv)

Contains personal demographic information including names, contact details, gender, and age data for 10,000 individuals.

5.3 Organisations Dataset (organizations-10000.csv)

Contains company-level data including organisation name, industry, number of employees, and contact information for 10,000 organisations.

6. System Architecture & Data Flow

The project follows a linear Extract-Transform-Load (ETL) architecture, illustrated in the flow below:

1. Data Ingestion

PySpark reads the three CSV files from local disk using spark.read.csv() with header=True. Spark automatically infers column names from the header row.

2. Data Cleaning & Transformation

Duplicate rows are removed from all three DataFrames using dropDuplicates(). The Subscription Date column is parsed from a string to a proper DateType using to_date() with the format pattern 'yyyy-MM-dd', producing a new column subscription_date_fixed.

3. Exploratory Summary (Spark)

Record counts for each dataset are printed to confirm data integrity after the cleaning step.

4. MongoDB Connection

A MongoClient instance is created using the MongoDB Atlas SRV connection string pointing to Cluster0. Three collection handles are obtained: customers, people, and organizations inside the spark_mango database.

5. Serialisation to JSON

Each Spark DataFrame is converted to a list of JSON strings via toJSON().collect(). The collected strings are then parsed back into Python dictionaries.

6. MongoDB Ingestion

Existing documents in each collection are cleared with delete_many({}) to ensure idempotent re-runs. The cleaned data is bulk-inserted using insert_many().

7. Aggregation Queries

MongoDB aggregation pipelines are run directly against the Atlas cluster to derive analytical insights (see Section 8).

8. Visualisation

Query results are plotted as bar charts, a pie chart, and a line chart using Matplotlib and Seaborn within the Jupyter Notebook.

9. Session Cleanup

The Spark session is gracefully shut down with spark.stop() to release cluster resources.

7. Source Code Walkthrough

The primary implementation is contained in spark-py.py (118 lines). The Jupyter Notebook connect.ipynb mirrors this logic with additional inline visualisation cells. The following sub-sections describe each logical block of the script.

7.1 Imports and Dependencies

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date
from pymongo import MongoClient
import json
```

PySpark provides the distributed computation engine. PyMongo is the official Python driver for MongoDB. The json module handles serialisation between PySpark's JSON output and Python dictionaries.

7.2 Spark Session Initialisation

```
spark = SparkSession.builder.appName('SparkMongoProject').getOrCreate()
```

A SparkSession is the unified entry point for Spark functionality. appName() assigns a human-readable name visible in the Spark UI. getOrCreate() reuses an existing session if one is active.

7.3 Data Loading

```
customers = spark.read.csv('customers-10000.csv', header=True)
people = spark.read.csv('people-10000.csv', header=True)
organizations = spark.read.csv('organizations-10000.csv', header=True)
```

spark.read.csv() returns a Spark DataFrame with schema inferred from headers. All column types default to StringType unless explicitly cast.

7.4 Data Cleaning

```
customers = customers.dropDuplicates()
people = people.dropDuplicates()
organizations = organizations.dropDuplicates()

customers = customers.withColumn(
'subscription_date_fixed',
to_date(col('Subscription Date'), 'yyyy-MM-dd')
)
```

dropDuplicates() performs a full-row deduplication, retaining only unique records. withColumn() adds the parsed date column non-destructively alongside the original string column, preserving backward compatibility.

7.5 MongoDB Connection & Ingestion

```
client =
MongoClient('mongodb+srv://<user>:<password>@cluster0.fltnplm.mongodb.net/')
db = client['spark_mango']

customers_col = db['customers']
```

```
people_col = db['people']
organizations_col = db['organizations']

customers_json = customers.toJSON().collect()
customers_col.delete_many({})
customers_col.insert_many([json.loads(row) for row in customers_json])
```

toJSON() serialises each DataFrame row to a JSON string using Spark's internal JSON encoder. collect() materialises the distributed dataset on the driver node. delete_many({}) with an empty filter document removes all existing records, making each pipeline run fully idempotent. insert_many() performs a bulk write for optimal throughput.

8. MongoDB Aggregation Queries

Four aggregation queries are executed against the customers collection. Each uses the MongoDB Aggregation Pipeline framework, which processes documents through a sequence of transformation stages.

8.1 Customers per Country

```
customers_col.aggregate([
{'$group': {'_id': '$Country', 'count': {'$sum': 1}}},
{'$sort': {'count': -1}}
])
```

Groups all customer documents by the Country field and counts the number of customers in each country. The results are sorted in descending order of count, providing a full frequency distribution across all represented countries.

8.2 Top 5 Countries

```
customers_col.aggregate([
{'$group': {'_id': '$Country', 'count': {'$sum': 1}}},
{'$sort': {'count': -1}},
{'$limit': 5}
])
```

Extends the previous query with a $limit stage to return only the five countries with the highest number of customers. This is the query whose result feeds the primary bar chart visualisation.

8.3 Customers per Company

```
customers_col.aggregate([
{'$group': {'_id': '$Company', 'count': {'$sum': 1}}},
{'$sort': {'count': -1}}
])
```

Groups customer records by the Company field to reveal how customers are distributed across different organisations. The output drives the second bar chart visualisation.

8.4 Subscription Date Range

```
customers_col.aggregate([
{'$group': {
'_id': None,
'earliest': {'$min': '$Subscription Date'},
'latest': {'$max': '$Subscription Date'}
}}
])
```

Computes the earliest and latest subscription dates across the entire collection in a single pass, using $min and $max accumulators. The resulting date range informs the subscription trend line chart.

9. Visualisations

The Jupyter Notebook (connect.ipynb) contains all visualisation cells. Four chart types were produced from the analytical query results:

Chart	File	Data Source	Purpose
Bar Chart — Country	Bar Chart.jpeg	Query 8.1 / 8.2 results	Shows the count of customers per country; top-5
Bar Chart — Company	Bar Chart (2).jpeg	Query 8.3 results	Shows distribution of customers across companie
Pie Chart	Pie chart.jpeg	People dataset (gender field)	Shows proportional gender distribution across the
Line Chart	Line Chart.jpeg	Query 8.4 / subscription dates	Shows customer subscription volume over time a

10. Setup and Execution Guide

10.1 Prerequisites

- Python 3.8 or later installed on the host machine.
- Java Development Kit (JDK) 8 or 11 — required by Apache Spark.
- Apache Spark installed and SPARK_HOME environment variable configured.
- An active MongoDB Atlas account with a cluster provisioned.
- Network access to MongoDB Atlas (IP whitelisting configured in Atlas).

10.2 Installation

```
# Clone the repository
git clone https://github.com/mohamedtarek750/Spark-MongoDB-Project.git
cd Spark-MongoDB-Project

# Install Python dependencies
pip install -r requirements.txt
```

10.3 Configuration

Open spark-py.py and update the following values before running:

```
# Update dataset paths to match your local filesystem
customers = spark.read.csv('/path/to/customers-10000.csv', header=True)
people = spark.read.csv('/path/to/people-10000.csv', header=True)
organizations = spark.read.csv('/path/to/organizations-10000.csv', header=True)

# Replace with your MongoDB Atlas connection string
client =
MongoClient('mongodb+srv://<username>:<password>@<cluster>.mongodb.net/')
```

10.4 Running the Script

```
# Execute the main pipeline script
python spark-py.py

# Or run the interactive notebook
jupyter notebook connect.ipynb
```

Successful execution prints the record counts for each dataset, confirms successful MongoDB insertion, and outputs the aggregation query results to the console.

11. Design Decisions & Rationale

Why Apache Spark?

Apache Spark provides a distributed, in-memory computing framework that handles datasets far larger than available RAM by processing data in partitioned, parallel fashion. Even for the 10,000-record datasets used here, Spark demonstrates the scalable patterns that would apply to production-scale data volumes.

Why MongoDB Atlas?

MongoDB Atlas is a fully managed cloud database service that eliminates the operational overhead of self-hosting. Its document model maps naturally to the JSON-serialised Spark DataFrame rows, making the ingestion step straightforward without requiring schema definition or DDL statements.

Why toJSON().collect() for ingestion?

PySpark does not have a native MongoDB connector configured in this project. The toJSON().collect() approach brings the processed data to the Spark driver as a Python list, which PyMongo then bulk-inserts. This is acceptable for datasets of this size. For larger datasets, the mongo-spark-connector would be preferred to avoid collecting data on the driver.

Why delete_many({}) before insert_many()?

This pattern ensures idempotency: running the script multiple times does not result in duplicate documents. Each execution starts from a clean state, which is important during development and testing.

Why a separate Jupyter Notebook?

The notebook (connect.ipynb) provides an interactive, cell-by-cell execution environment with inline chart rendering, making it suitable for exploration and presentation. The standalone Python script (spark-py.py) is suitable for automated or scheduled execution.

12. Limitations & Future Improvements

12.1 Current Limitations

- Hard-coded local file paths in spark-py.py require manual updates per machine.
- Credentials (MongoDB connection string) are embedded in source code — a security risk for production use. Environment variables or a secrets manager should be used instead.
- toJSON().collect() loads all data onto the driver; this approach does not scale to very large datasets.
- No null value handling is implemented beyond deduplication — fields with missing values are not imputed or flagged.
- The Spark session runs in local mode (no distributed cluster configured).

12.2 Recommended Improvements

- Use environment variables or a .env file for all configurable parameters (paths, credentials).
- Integrate the Mongo Spark Connector (org.mongodb.spark:mongo-spark-connector) for direct DataFrame writes to MongoDB, bypassing the collect() bottleneck.
- Add explicit null handling: drop rows with critical null fields or impute defaults.
- Implement schema enforcement using StructType to catch data type mismatches early.
- Deploy to a Spark cluster (e.g., Databricks, AWS EMR) to validate true distributed execution.
- Add unit tests for transformation logic using PySpark's built-in testing utilities.
- Automate execution with Apache Airflow or a cron job for scheduled pipeline runs.

13. Conclusion

This project successfully demonstrates a foundational data engineering pipeline using Apache Spark and MongoDB Atlas. The team implemented all core stages of an ETL workflow: data ingestion from CSV files, distributed processing and transformation with PySpark, cloud-based document storage with MongoDB, analytical querying via aggregation pipelines, and visual exploration through multiple chart types.

The codebase is clean, well-structured, and logically segmented into discrete functional blocks. While several areas — such as credential management and scalability of the ingestion strategy — offer scope for improvement, the project meets its academic objectives and provides a solid practical foundation for more advanced Big Data engineering work.

14. References

- Apache Spark Documentation — https://spark.apache.org/docs/latest/
- PySpark API Reference — https://spark.apache.org/docs/latest/api/python/
- MongoDB Atlas Documentation — https://www.mongodb.com/docs/atlas/

- PyMongo Documentation — https://pymongo.readthedocs.io/
- Project Repository — https://github.com/mohamedtarek750/Spark-MongoDB-Project